SENIOR INTERVIEW · QUICK REFERENCE

ASP.NET Core Controllers Cheat Sheet

`2xx` success `3xx` redirect `4xx` client error `5xx` server error

HTTP verbs

Verb	Use it to	Idem.	Safe	Success	Typical errors
GET	Read one item or a list	✅	✅	`200` (`304` if cached)	`404` `400`
POST	Create, or run a command	❌	❌	`201` `202` `200`	`400` `409` `422`
PUT	Replace the whole resource (or upsert)	✅	❌	`204` `200` `201`	`400` `404` `409` `412`
PATCH	Change some fields	❌	❌	`204` `200`	`400` `404` `409` `422`
DELETE	Remove	✅	❌	`204`	`404` `409`
HEAD / OPTIONS	Headers only / allowed methods, CORS	✅	✅	`200` `204`	—

Idempotent: N calls leave the same state as 1 call (PUT, DELETE). POST is not.

Safe: doesn't change server state (GET, HEAD, OPTIONS).

PUT = here is the full new version. **PATCH** = here are the changes.

2xx success / 3xx redirect

Code	Helper	When
`200` OK	`Ok(dto)` / `return dto;`	Read worked, or update returns a body
`201` Created	`CreatedAtAction(nameof(Get), new {id}, dto)`	POST created it. Sends `Location`
`202` Accepted	`AcceptedAtAction(...)`	Queued, finishes later
`204` No Content	`NoContent()`	PUT / PATCH / DELETE, nothing to return
`301` / `308`	`RedirectPermanent` / `...PreserveMethod`	Moved for good (308 keeps method + body)
`302` / `307`	`Redirect`, `LocalRedirect` / `RedirectPreserveMethod`	Temporary. `LocalRedirect` blocks open redirects
`304` Not Modified	`StatusCode(304)`	`If-None-Match` equals the ETag

4xx client error / 5xx server error

Code	Helper	When
`400` Bad Request	automatic, `ValidationProblem()`	Bad input / validation failed
`401` Unauthorized	`Unauthorized()`	Not logged in, bad or expired token
`403` Forbidden	`Forbid()`	Logged in but not allowed
`404` Not Found	`NotFound()`	That resource doesn't exist
`409` Conflict	`Conflict()`	Duplicate, wrong state, concurrency clash
`412` Precondition Failed	`StatusCode(412)`	`If-Match` ETag is stale
`415` Unsupported Media	framework	Wrong `Content-Type`
`422` Unprocessable	`UnprocessableEntity()`	Valid input, business rule fails
`429` Too Many Requests	`StatusCode(429)`	Rate limited
`500` / `503`	`Problem()` from middleware	Unhandled exception / downstream down

The 6 rules to never forget

- POST that creates → `201` + `Location` via `CreatedAtAction`.
- Update or delete with nothing to return → `204` `NoContent()`.
- Missing resource → `404`. Search with no results → `200` + `[]`.
- Bad input → `400` (automatic with `[ApiController]`). Broken business rule → `422` or `409`.
- `401` = who are you? `403` = I know you, and no.
- Never leak exceptions: map them once in middleware and return **ProblemDetails**.

Pick between look-alikes

Pair	Difference
`400` vs `422`	400 = malformed / invalid input. 422 = well-formed, breaks a business rule.
`409` vs `412`	409 = conflicts with current state. 412 = `If-Match` precondition failed.
`200` vs `204`	Return the body (200) or nothing (204) after an update.
`201` vs `202`	Created now (201) vs accepted, done later (202).
`302` vs `307`	307 keeps the HTTP method and body; 302 may turn POST into GET.

Routing

```
[ApiController]              // auto 400 + ProblemDetails
[Route("api/[controller]")]  // => "api/products"
public class ProductsController : ControllerBase { }
```

Attribute	Matches
`[HttpGet]`	`GET /api/products?page=1&take=10`
`[HttpGet("{id:int}")]`	`GET /api/products/5`
`[HttpGet("{id:int}/stock")]`	`GET /api/products/5/stock`
`[HttpGet("search")]`	`GET /api/products/search?name=tv`
`[HttpGet("~/health")]`	`GET /health` (`~` skips the prefix)
`[HttpGet("{id:int}", Name = "GetProduct")]`	named route for `CreatedAtRoute`

Constraints: `int` `long` `guid` `bool` `alpha` `min(1)` `max(9)` `range(1,9)` `minlength(3)` `regex(..)` · optional `{id?}` · default `{page=1}`

Constraint fails → `404` (no route matches). No constraint + `/products/abc` → `400` from model binding.

Binding sources

```
public IActionResult Example(
  [FromRoute]  int id,            // /products/{id}
  [FromQuery]  int page,          // ?page=2
  [FromQuery]  int[] ids,         // ?ids=1&ids=2
  [FromBody]   CreateProduct cmd, // JSON, one per action
  [FromHeader(Name = "If-Match")] string etag,
  [FromForm]   IFormFile file,    // multipart upload
  [FromServices] IClock clock)    // DI into the method
```

With `[ApiController]`: complex types → body. Simple types → route if the name matches, else query.

Return type	Verdict
`Task<ProductDto>`	Always `200`. **Can't return 404.** (repo today)
`Task<IActionResult>`	Any code, but Swagger can't see the schema
`Task<ActionResult<ProductDto>>`	**Use this.** Return `dto` → 200, or `NotFound()`

Document each response for Swagger

```
[ProducesResponseType(typeof(ProductDto), 200)]
[ProducesResponseType(404)]
```

Full CRUD controller (CQRS + MediatR)

```
[ApiController]
[Route("api/v1/products")]
public class ProductsController : ControllerBase
{
  private readonly IMediator _mediator;
  private readonly IProductQueryService _q;
  public ProductsController(IMediator m, IProductQueryService q)
    => (_mediator, _q) = (m, q);

  // 200 · 400
  [HttpGet]
  public async Task<ActionResult<DataCollection<ProductDto>>>
    GetAll(int page = 1, int take = 10,
           CancellationToken ct = default)
  {
    if (page < 1 || take < 1 || take > 100) return BadRequest();
    return Ok(await _q.GetAllAsync(page, take, ct));
  }

  // 200 · 404   (use FirstOrDefault, NOT Single => no 500)
  [HttpGet("{id:int}")]
  public async Task<ActionResult<ProductDto>>
    GetById(int id, CancellationToken ct)
  {
    var dto = await _q.FindAsync(id, ct);
    return dto is null ? NotFound() : dto;
  }

  // 201 + Location · 400 · 409
  [HttpPost]
  public async Task<ActionResult<ProductDto>>
    Create(CreateProduct cmd, CancellationToken ct)
  {
    var id = await _mediator.Send(cmd, ct); // IRequest<int>
    var dto = await _q.FindAsync(id, ct);
    return CreatedAtAction(nameof(GetById), new { id }, dto);
  }

  // 204 · 400 · 404
  [HttpPut("{id:int}")]
  public async Task<IActionResult>
    Update(int id, UpdateProduct cmd, CancellationToken ct)
  {
    if (id != cmd.Id) return BadRequest("Id mismatch");
    var found = await _mediator.Send(cmd, ct);
    return found ? NoContent() : NotFound();
  }

  // 204 · 400 · 404 · 415 (needs AddNewtonsoftJson)
  [HttpPatch("{id:int}")]
  [Consumes("application/json-patch+json")]
  public async Task<IActionResult> Patch(int id,
    JsonPatchDocument<ProductPatch> patch, CancellationToken ct)
  {
    var model = await _q.FindForPatchAsync(id, ct);
    if (model is null) return NotFound();
    patch.ApplyTo(model, ModelState);
    if (!TryValidateModel(model))
      return ValidationProblem(ModelState);
    await _mediator.Send(new SaveProduct(id, model), ct);
    return NoContent();
  }

  // 204 · 404
  [HttpDelete("{id:int}")]
  public async Task<IActionResult>
    Delete(int id, CancellationToken ct)
  {
    var deleted = await _mediator.Send(new DeleteProduct(id), ct);
    return deleted ? NoContent() : NotFound();
  }
}

// Command that returns the new id
public class CreateProduct : IRequest<int>
{
  [Required, StringLength(100)] public string Name { get; set; }
  [Range(0.01, 1_000_000)] public decimal Price { get; set; }
}
```

	`Publish(INotification)` (repo today)	`Send(IRequest<T>)`
Handlers / returns	0..N handlers, returns nothing	exactly 1 handler, returns `T`
Use for	domain events: something happened	commands and queries; gives the id for a `201`

Validation and ProblemDetails

With `[ApiController]`, a failed DataAnnotation returns this **before your action runs**:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": { "Name": ["The Name field is required."] }
}
```

Manual errors

```
ModelState.AddModelError("Price", "Must be > 0");
return ValidationProblem(ModelState);   // 400

// any status code with a ProblemDetails body
return Problem(statusCode: 422,
  title: "Not enough stock", detail: $"Product {id}");
```

Senior bonus: **FluentValidation** `AbstractValidator<T>`, or a MediatR `IPipelineBehavior`.

Global exception middleware

```
try { await _next(ctx); }
catch (Exception ex)
{
  var (status, title) = ex switch
  {
    NotFoundException            => (404, "Not found"),
    BusinessRuleException        => (422, ex.Message),
    DbUpdateConcurrencyException => (409, "Conflict"),
    UnauthorizedAccessException  => (403, "Forbidden"),
    _                            => (500, "Unexpected")
  };
  if (status == 500) _log.LogError(ex, "Unhandled");
  ctx.Response.StatusCode = status;
  ctx.Response.ContentType = "application/problem+json";
  await ctx.Response.WriteAsJsonAsync(new ProblemDetails {
    Status = status, Title = title,
    Instance = ctx.Request.Path });
}
// register early: app.UseMiddleware<ExceptionMiddleware>();
```

Throw typed domain exceptions from handlers, never `new Exception("...")` (that becomes a `500`).

Special scenarios

Scenario	Code	How
Long job (report, reindex)	`202`	`AcceptedAtAction(nameof(Status), new { id }, job)`; client polls
Cache check with ETag	`304`	`If-None-Match == etag` → `StatusCode(304)`; else 200 + `ETag` header
Lost-update protection	`412`	`If-Match` ≠ current ETag → 412; missing header → `428`
Retry-safe POST	`201`	`Idempotency-Key` header: same key replays the first response
Duplicate key on create	`409`	unique index + catch `DbUpdateException` → `Conflict()`
Soft-deleted resource	`410`	`IsDeleted` → `Problem(statusCode: 410)` (it existed before)
File upload	`415`	`IFormFile` + `[RequestSizeLimit]`; wrong type → 415
Paged list	`200`	`DataCollection<T>` + `X-Total-Count`; page past the end → `[]`
Moved endpoint	`301`	`RedirectToActionPermanent(nameof(GetById), new { id })`

Auth: 401 vs 403

```
[Authorize]                  // must be logged in
public class OrdersController : ControllerBase
{
  [AllowAnonymous] [HttpGet] ...            // public
  [Authorize(Roles = "Admin")] [HttpDelete] ...
  [Authorize(Policy = "CanManageStock")]    // preferred
  [HttpPut] ...
}
```

`401` no / bad token → client should log in. `403` valid user, missing role or policy.

Order: `UseRouting` → `UseAuthentication` → `UseAuthorization` → `UseEndpoints`.

DI lifetimes: Transient = new each time · Scoped = per request (DbContext) · Singleton = one per app. Never inject Scoped into Singleton.

This repo: today → senior fix

Today	Fix
`GetAsyc` uses `SingleAsync`: missing id → `500`	`FirstOrDefaultAsync` + `NotFound()` → `404`
`POST /products`: `Publish` + `Ok()` → `200`, no id	`Send` + `CreatedAtAction` → `201` + `Location`
`?ids=1,abc` + `Convert.ToInt32` → `500`	`[FromQuery] int[] ids` → automatic `400`
Stock handler throws `System.Exception` → `500`	`BusinessRuleException` → `422` / `409` via middleware
`[HttpGet("{id}")]` with no constraint	`{id:int}` → `404` for `/products/abc`
No validation, no `CancellationToken`	DataAnnotations / FluentValidation; pass `ct` to EF and MediatR

Interview one-liners

- **`ControllerBase` not `Controller`:** APIs don't need views.
- **`[ApiController]` gives:** attribute routing, auto `400`, inferred `[FromBody]`, ProblemDetails.
- **POST vs PUT:** server picks the URI vs client knows it; PUT is idempotent.
- **What's in a `201`:** `Location` header + the created body.
- **Business rules live** in handlers / domain. Controllers: bind → send → map to a code.
- **Why async + `CancellationToken`:** frees threads; stops work when the client leaves.
- **DTO, not entity:** no schema leaks, no cycles, no over-posting.
- **Versioning:** `[ApiVersion("2.0")]` + `api/v{version:apiVersion}/...`.
- **Minimal APIs (.NET 6+):** `app.MapGet("/{id:int}", ...)` returning `TypedResults.Ok/NotFound`.